

## Lab 5

### Q.1 Algorithms using Backtracking approach.

#### Introduction:

Backtracking is a systematic method for exploring all possible configurations of a problem to find a solution that satisfies specific constraints. It builds the solution candidate by candidate and abandons a candidate ("backtracks") as soon as it determines that the candidate cannot possibly be completed to a valid solution. This approach is particularly effective for combinatorial problems like N-Queen's puzzle.

#### Objectives:

- To understand the recursive nature of backtracking algorithms.
- To understand and implement N queen puzzle.
- To analyze the performance and state-space tree of the N-Queens problem.

#### Theory:

The N-Queens Problem requires placing N-queens on an  $N \times N$  chessboard such that no two queens attack each other.

- Vertical/Horizontal Constraint: No two queens can be in same row or column.
- Diagonal Constraint: No two queens can be on the same diagonal. For a queen at  $(x, c)$ , any other queen at  $(x', c')$  is on the same diagonal if  $|x - x'| = |c - c'|$

Backtracking logic: We place a queen in the first available square of a column and move to the next column. If we reach a column where no square is safe, we return to the previous column and move that queen to the next possible valid square.

### Algorithm:

- i) Start in the leftmost column (column 0).
- ii) If all queens are placed (column =  $n$ ), print the board and return true.
- iii) Try all rows in the current column. For each row:
  - a. Check if placing a queen at `board[row][col]` is safe.
  - b. If safe, mark `board[row][col] = 1`.
  - c. Recursively try to place the rest of the queens in the next column.
  - d. If recursion returns true, a solution is found.
  - e. If recursion returns false, unmark `board[row][col] = 0` (Backtrack).
- iv) If no row works, return false to the previous recursive call.

### Time and Space Complexity:

- Time complexity:  $O(N!)$ . For each column, no. of available choices decrease, similar to a factorial growth.
- Space complexity:  $O(N^2)$  for the 2D array &  $O(N)$  for the recursion stack.

Code:

```
#include <iostream>
using namespace std;

#define N 4

void printBoard (int board[N][N])
{
    for (int i=0; i<N; i++)
    {
        for (int j=0; j<N; j++)
        {
            cout << (board[i][j] ? "Q " : ". ");
        }
    }
}

bool isSafe (int board[N][N], int row, int col)
{
    int i, j;
    // Check row on left side
    for (i=0; i<col; i++)
        if (board[row][i])
            return false;

    // Check upper diagonal on left
    for (i=row, j=col; i>=0 && j>=0; i--, j--)
        if (board[i][j])
            return false;

    // Check lower diagonal on left side
    for (i=row, j=col; j>=0 && i<N; i++, j++)
        if (board[i][j])
            return false;

    return true;
}
```

```
bool solveNQ (int board[N][N], int col)
```

```
{
```

```
    if (col >= N)
```

```
        return false, true;
```

```
    for (int i = 0; i < N; i++)
```

```
    {
```

```
        if (isSafe(board, i, col))
```

```
        {
```

```
            board[i][col] = 1;
```

```
            if (solveNQ(board, col+1))
```

```
                return true;
```

```
            board[i][col] = 0; // Backtrack
```

```
        }
```

```
    }
```

```
    return false;
```

```
}
```

```
int main()
```

```
{
```

```
    int board[N][N] = {0};
```

```
    if (solveNQ(board, 0))
```

```
    {
```

```
        printBoard(board);
```

```
    }
```

```
    else
```

```
    {
```

```
        cout << "No solution exists";
```

```
    }
```

```
    return 0;
```

```
}
```

### Output:

• • Q •  
Q • • •  
• • • Q  
• Q • •

### Conclusion:

The N-Queens problem was successfully solved using the backtracking approach. The experiment highlights how recursion can be used to explore a decision tree and how backtracking optimizes the search by abandoning invalid paths early. This method ensures an exact solution is found while keeping the memory usage relatively low.

### References:

- E. Horowitz and S. Sahni, "Foundational Computer Algorithms".
- R. Sedgewick, "Algorithms in C++".

## Q. Implement approximation Algorithm.

### Introduction:

An approximate algorithm is a way of approach NP-completeness for the optimization problem. This technique does not guarantee the best solution. The goal of an approximation algorithm is to come as close as possible to the optimum value in a reasonable amount of time which is at the most polynomial time.

### Objective:

- To implement the vertex cover problem.
- To understand the limitations of solving NP-complete problems.
- To analyze the time efficiency of approximation.

### Theory:

A vertex cover problem is a classic optimization problem in graph theory and is known to be NP-complete, meaning no polynomial-time algorithm is known to find the absolute minimum vertex cover. For vertex cover, a simple greedy strategy can find a cover that is at most twice the size of the optimal minimum cover.

A vertex cover is a set of vertices  $S$  such that every edge  $(u, v)$  in the graph has at least one endpoint in  $S$ .

- The 2-Approximation Strategy:

- i) Pick an edge  $(u, v)$  that hasn't been covered yet
- ii) Add both  $u$  and  $v$  to the vertex cover set.
- iii) Ignore all other edges connected to  $u$  or  $v$ .

Algorithm:

- i) Initialize an array of size  $V$  to all zeros (false).
- ii) Loop through every vertex  $u$  from  $0$  to  $V-1$ :
- iii) If vertex  $u$  is not yet "visited":
  - a. Look at all its neighbours  $v$ .
  - b. If a neighbour  $v$  is also not "visited":
    - Mark both  $u$  and  $v$  as visited (included in cover).
    - Break the inner loop to move to the next uncovered vertex.
- iv) Print all vertices that are marked as visited.

Code:

```
#include <iostream>
using namespace std;
#define V 6

void findVertexCover (int graph[V][V])
{
    bool visited[V] = {false};

    for (int u = 0; u < V; u++)
    {
        if (!visited[u])
        {
            for (int v = 0; v < V; v++)
            {
                if (graph[u][v] && !visited[v])
                {
                    visited[u] = true;
                    visited[v] = true;
                    break;
                }
            }
        }
    }
}
```

```

cout << "Approximate Vertex cover: ";
for (int i = 0; i < V; i++)
{
    if (visited[i])
        cout << i << " ";
}
cout << endl;
}

int main()
{
    // Adjacency matrix representation
    int graph[V][V] = {
        {0, 1, 1, 0, 0, 0},
        {1, 0, 0, 1, 0, 0},
        {1, 0, 0, 0, 0, 1},
        {0, 1, 0, 0, 1, 0},
        {0, 0, 0, 1, 0, 1},
        {0, 0, 1, 0, 1, 0}
    };

    findVertexCover(graph);
    return 0;
}

```

Output:

Approximate Vertex cover: 0 1 2 3 4 5

## Time and Space Complexity:

- Time complexity:  $O(V^2)$  when using an adjacency matrix, as we check all possible edge pairs. (Can be  $O(V+E)$  with adjacency lists).
- Space complexity:  $O(V^2)$  to store the graph in a matrix and  $O(V)$  for visited array.

## Conclusion:

The implementation of the vertex cover approximation shows that for computationally difficult problems, a greedy approach can provide a reliable solution in linear or quadratic time.

## Reference:

1. S.S. Skiena, "The algorithm Design Manual".
2. T.H. Cormen, "Introduction to Algorithms".